HYPERLOGISTICS-STREAM: REAL-TIME HUB-TO-HUB ETA

Tony Nguyen, Joe Doan and Menkal Son COMP-SCI 5540 — Principles of Big Data Management — Fall 2026

UMKC

KEY

- **Data Source / Store** — BTS, NOAA, FAA, NWS, OpenSky; Parquet tiers
- **Kafka Ingestion** — Replay and live topics
- **Spark Streaming** — Weather join, repeat-safe writes
- **ML Model** — GBT vs. two baselines
- **Dashboard** — Streamlit, 3 tabs

Problem Statement

- Unexpected transit delays bottleneck dock doors and downstream **hubs**.
- Static schedules cannot react to weather, airport ground stops or a backlog building up.
- **Goal:** predict arrival-delay change (Δ = ArrDelay − DepDelay) per trip in **real time**, and show how a hub compares with its **typical** day.

Dataset Description

BTS On-Time Performance
2018–2024 flights, 24 US hubs, ≥11 GB. Train 8.0M, validate 1.67M (2024).

NOAA ISD Weather
Hourly wind, visibility, ceiling, temperature at 40 stations.

FAA NAS Status
Live airport delays and closures.

NWS Weather
Live METAR observations per hub.

OpenSky Network
Live aircraft positions for the radar map.

Baseline and features

Typical-delay table: 24 hubs × local hour × weekday (3,878 cells, training years only) powers "today vs. typical".

14 model features: departure delay, distance, dep./arr. hour (UTC), weekday, lane mean delay, plus 4 destination and 4 origin weather fields.

Honest split: trained on 2018–2023 except 2020, evaluated on 2024, which the model never saw.

Layer 1: **INGESTION** (Apache Kafka)

Replay producer
BTS 2024 trips, 1k / 5k / 20k events/s

Live producer
OpenSky, FAA NAS, NWS METAR

→ **Kafka topics**
flights_stream, live_flights, airport_status, weather

→ **Record and replay**
live sessions saved as JSONL, so the demo never depends on API uptime

Layer 2: **STREAM PROCESSING & STORAGE** (Medallion Architecture)

Spark Structured Streaming

1 Parse & filter
2 NOAA weather join
3 Dedupe on flight_id
4 GBT scoring

Replaying a batch adds 0 rows to Gold (tested)

→ **Bronze** raw Kafka records → **Silver** cleaned, weather joined → **Gold** ETAs, hub congestion

Layer 3: **INTELLIGENCE & EVALUATION** (Spark MLlib)

Baseline 1
no change (Δ = 0)

Baseline 2
training lane-mean delay

Typical-delay table
hub × hour × weekday

→ **GBTRegressor, 14 features**

10.53 min MAE on 2024 (no change: 12.75)	**17%** better than no change
+1.0% from weather (ablation)	**2.5%** better than lane mean

Lane and schedule matter most.

→ **Error analysis**
Hardest hub: JFK (14.0 min MAE). Easiest: PHX (8.9).
Flights >15 min late are under-predicted by ~34 min.

Layer 4: **SERVING & DASHBOARD**

FastAPI
hub congestion and ETA queries

→

Live Radar
Hub map with FAA delays, weather, aircraft, and today vs. typical.

Replay & Benchmark
Fleet map, hub pressure, weather impact, worst lanes, system health.

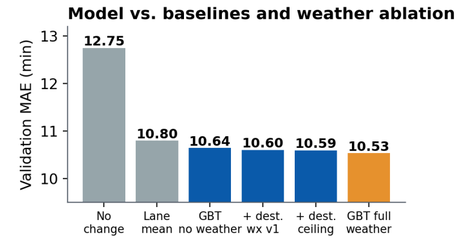
Model Insights
Feature groups, errors by hub and hour, seasonality.

Demo line: *"DFW is running N min worse than typical."* Predictions are always labelled as predictions.

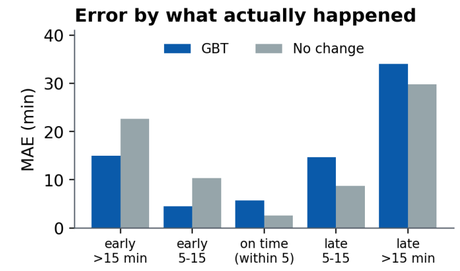
Demo

github.com/mosomo82/HyperLogistic_Stream

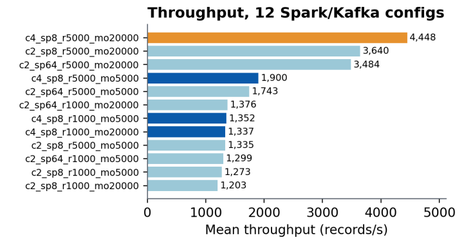
Results & Evaluation



Weather adds about 1%. MAE on 2024: GBT 10.53, no change 12.75, lane mean 10.80.



Wins on early flights, loses to "no change" on on-time and late ones. Not a severe-delay alarm.



4,448 records/s best (orange); 4 cores (dark) beat 2 by ~22%. Provisional: 4 batches per config.

Conclusion & Future Work

- Kafka → Spark → Medallion → dashboard, with **repeat-safe** writes and **23 automated tests** in CI. Big delays are still missed.
- Next: severe-delay classifier, full benchmark re-run, real freight data.